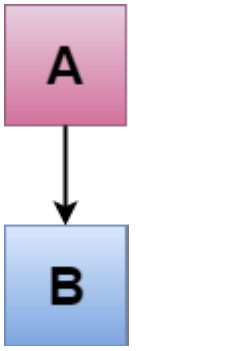
CORE–3: PROGRAMMING USING C++ (UNIT – 3)

Inheritance: inheritance is a process in which one object acquires the properties and behaviours of its parent object. The class which inherits the members of another class is called **derived class** and the class whose members are inherited is called **base class**. The main advantage of inheritance is **code reusability i.e.** we can reuse the members of the parent class. So, there is no need to define the member again.

Types of Inheritance: C++ supports five types of inheritance:

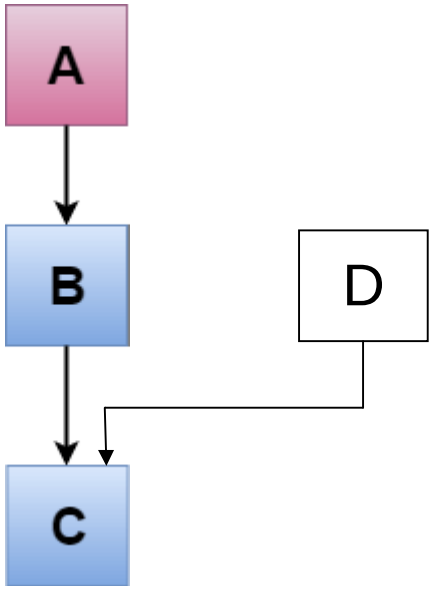
- Single inheritance
- Multilevel inheritance
- Multiple inheritances
- Hierarchical inheritance
- Hybrid inheritance



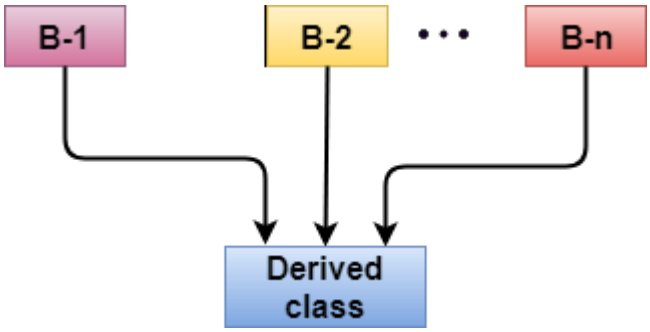
(Single inheritance)



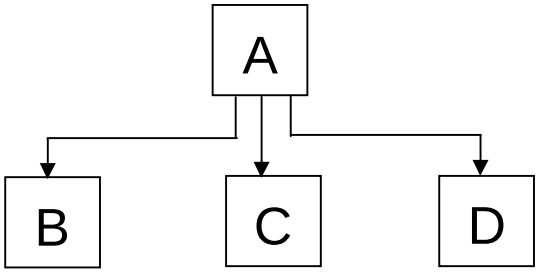
(Multilevel inheritance)



(Hybrid inheritance)



(Multiple inheritance)



(Hierarchical inheritance)

The Syntax of Derived class:

```
class derived_class_name : visibility-mode base_class_name  {
   // body of the derived class.  };
```

Where, derived_class_name is the name of the derived class.
visibility mode specifies how the base class is inherited. It can be public, private or protected.
base_class_name is the name of the base class.

- When the base class is privately inherited by the derived class, public members of the base class become the private members of the derived class. Therefore, the public members of the base class are not accessible by the objects of the derived class. They are accessible only by the member functions of the derived class.
- When the base class is publicly inherited by the derived class, public members of the base class also become the public members of the derived class. Therefore, the public members of the base class are also accessible by the objects of the derived class.
- The default mode of visibility is private.
- The private members of the base class are never inherited.

Single Inheritance: Single inheritance is defined as the inheritance in which a derived class is inherited from the only one base class. For example,

```
#include<iostream>
using namespace std;
class A {
  public:   int a;
    void showA() { cout<<a<<endl; }   };
class B: public A {
  public:  int b;
   void showB() { cout<<b<<endl; }   };
```

```
int main() {
    B ob;        ob.a = 10; ob.b=20;
    ob.showA();        ob.showB();
    return 0;
}
Output:
10
20
```

Making a Private Member Inheritable: The private members are not inheritable. If we modify the visibility label by making it public, this takes away the advantage of data hiding. C++ introduces a third visibility modifier, i.e., **protected**. The member which is declared as protected will be accessible to all the member functions within the class as well as the classess derived from it.

Visibility modes can be classified into three categories:

- **Public**: When the member is declared as public, it is accessible to all the functions of the program.
- **Private**: When the member is declared as private, it is accessible within the class only.
- **Protected**: When the member is declared as protected, it is accessible within its own class as well as the class immediately derived from it.

Visibility of Inherited Members			
Base class visibility	**Derived class visibility (Visibility mode)**		
	Public derivation	**Private derivation**	**Protected derivation**
Private	Not Inherited	Not Inherited	Not Inherited
Protected	Protected	Private	Protected
Public	Public	Private	Protected

Multilevel Inheritance: Multilevel Inheritance is a process of deriving a class from another derived class. Example of multilevel inheritance is shown below.

```
#include<iostream>
using namespace std;

    class A {    public:      int a;      };
    class B: public A   {   public:      int b;      };
    class C: public B   {   public:      int c;      };
```

```
int main(void) {
      C oc;
      oc.a=10; oc.b=20;  oc.c=30;
      cout<<oc.a<<"\t"<<oc.b<<"\t"<<oc.c;
      return 0;
}
```

Output: 10 20 30

Multiple Inheritance: Multiple **inheritance** is the process of deriving a new class that inherits the attributes from two or more classes.

Syntax of the Derived class:

```
class D : visibility B-1, visibility B-2, .....
{
        // Body of the class;
};
```

Example

```
#include<iostream>
using namespace std;
class A  {     public:      int a;  };
class B  {     public:     int b;  };
class C : public A, public B  {
  public:
   void display()  {
      cout<<"Addition of a and b is : "<<a+b; }
 };
```

```
int main() {
   C oc;
   occ.a=10;
   oc.b=20;
   oc.display();
   return 0;
}
```

Output: Addition of a and b is : 30

Ambiguity Resolution in Inheritance: Ambiguity can be occurred in using the multiple inheritance when a function with the same name occurs in more than one base class. Let's understand this through an example:

```
#include<iostream>
using namespace std;
class A {
   public:
   void display(){
      cout << "Class A" << endl;
   }
};
class B  {
   public:
   void display(){
      cout << "Class B" << endl;
   }
};
class C : public A, public B  {
   void view() {
      display(); // Ambiguity
   }
};
```

```
int main()  {
    C oc;
    c.display();
    return 0;
}
```

Output:

```
error: reference to 'display' is ambiguous
       display();
```

The above issue can be resolved by using the class resolution operator with the function. In the above example, the derived class code can be rewritten as:

```
class C : public A, public B  {
   void view()     {
      A :: display(); // Calling the display() function of class A
      B :: display(); // Calling the display() function of class B
    }
};
```

Example showing function overriding:

```
class A  {
   public:
      void display()  {
        cout<<"Class A";  }
} ;
class B: public A  {
  public:
 void display()  {
      cout<<"Class B";  }
} ;
```

In this case, the function of the derived class overrides the method of the base class. Therefore, call to the display() function by the object of derived class will simply call the function defined in the derived class. If we want to invoke the base class function, we can use the class resolution operator.

```
int main()  {
    B ob;
   ob.display();          // Calling the display() function of B class.
   ob.A :: display();     // Calling the display() function of A class.
}
```

Hierarchical Inheritance: Hierarchical inheritance is defined as the process of deriving more than one class from a base class.

Syntax of Hierarchical inheritance:

```
class A  {      // body of the class A.  };
class B : public A   {      // body of class B.  };
class C : public A  {      // body of class C.  };
class D : public A  {      // body of class D.  };
```

Example of hierarchical inheritance:

```
#include<iostream>
using namespace std;
class Shape {
  public:
    int a, b;
    void get_data(int m,int n) {
        a= m;          b = n;
    }
};
class Rectangle : public Shape  {
    public:
    int rect_area() {
         return a*b;
    }
};
class Triangle : public Shape {
    public:
    int triangle_area()    {
        return 0.5*a*b;
    }
};
```

```
int main()  {
  Rectangle r;    Triangle t;
  int length,breadth,base,height;
  cout<<"Enter the length and breadth of a rectangle: \n";
  cin>>length>>breadth;
  r.get_data(length,breadth);
  cout<<"Area of the rectangle is : " << r.rect_area()<<endl;
  cout<<"Enter the base and height of the triangle: \n";
  cin>>base>>height;
  t.get_data(base,height);
  cout<<"Area of the triangle is : "<<t.triangle_area()<<endl;
  return 0;
}
```

Output:

```
Enter the length and breadth of a rectangle:
23
20
Area of the rectangle is : 460
Enter the base and height of the triangle:
2
5
Area of the triangle is : 5
```

Hybrid Inheritance: Hybrid inheritance is a combination of more than one type of inheritance. Let's see a simple example:

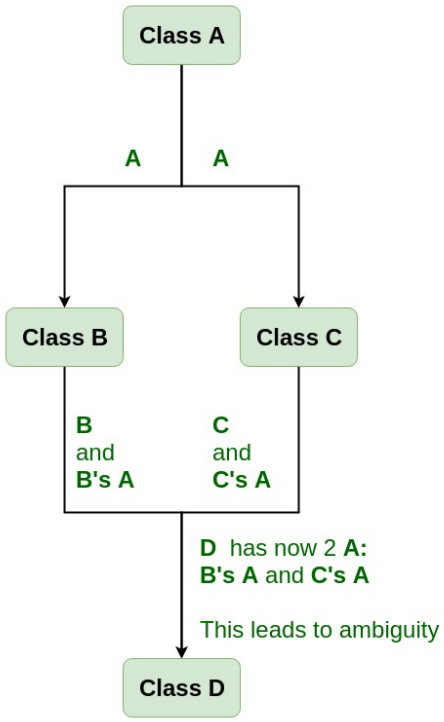
```
#include<iostream>
using namespace std;
class A  {
   protected:     int a;
   public:
   void get_a()     {
      cout << "Enter the value of 'a' : ";
      cin>>a;     }
};
class B : public A   {
   protected:  int b;
   public:
   void get_b()   {
      cout << "Enter the value of 'b' : ";
      cin>>b;     }
};
class C   {
   protected:  int c;
   public:
   void get_c()     {
      cout << "Enter the value of 'c' : ";
      cin>>c;     }
};
class D : public B, public C
{
   public:
   void mul()
   {
      get_a();
      get_b();
      get_c();
      cout << "Multiplication of a,b,c is : " <<a*b*c;
   }
};
int main()
{
   D d;
   d.mul();
   return 0;
}
```

Output:

```
Enter the value of 'a' : 10
Enter the value of 'b' : 20
Enter the value of 'c' : 30
Multiplication of a,b,c is : 6000
```

Virtual Base Classes: Virtual base classes are used to preventing multiple “instances” of a base class to be inherited to a derived class through multiple inheritances. Consider the situation where we have one class A .This class is A is inherited by two other classes B and C. Both these class are inherited into another in a new class D as shown in the figure.

As we can see from the figure that data members/function of class A are inherited twice to class D. One through class B and second through class C. When any data / function member of class A is accessed by an object of class D, ambiguity arises as to which data/function member would be called? One inherited through B or the other inherited through C. This confuses compiler and it displays error.



Example to illustrate virtual base class:

```
#include<iostream>
using namespace std;
class A {
public:
   void show()      {
      cout << "Hello form A \n";
   }
};
class B : public A {
};
class C : public A {
};
class D : public B, public C {
};
int main() {
   D od;
   od.show(); // Ambiguity error
   return 0;
}
```

To resolve this ambiguity when class **A** is inherited in both class **B** and class **C**, it is declared as **virtual base class** by placing a keyword **virtual** as :

Syntax for Virtual Base Classes:

Syntax 1:

```
class B : virtual public A { };
```

Syntax 2:

```
class C : public virtual A { };
```

Note: virtual can be written before or after the **public**.

Now only one copy of data/function member will be copied to class **C** and class **B** and class **A** becomes the virtual base class.

Virtual base classes offer a way to save space and avoid ambiguities in class hierarchies that use multiple inheritances. When a base class is specified as a virtual base, it can act as an indirect base more than once without duplication of its data members. A single copy of its data members is shared by all the base classes that use virtual base.

Abstract Classes: Sometimes implementation of all function cannot be provided in a base class because we don't know the implementation. Such a class is called abstract class. For example, let Shape be a base class. We cannot provide implementation of function draw() in Shape, but we know every derived class must have implementation of draw().

Characteristics of Abstract Class:

- Abstract Class is a class which contains at least one Pure Virtual function in it. Abstract classes are used to provide an Interface for its sub classes.
- Abstract class cannot be instantiated, but pointers and references of Abstract class type can be created.
- Abstract class can have normal functions, constructors and variables along with a pure virtual function.
- Abstract classes are mainly used as base classes, so that its derived classes can use its interface.
- Classes inheriting an Abstract Class must implement all pure virtual functions, or else they will become Abstract too.

Pure Virtual Functions: Pure virtual Functions are virtual functions with no definition. They start with virtual keyword and ends with = 0. Here is the syntax for a pure virtual function,

```
virtual void f() = 0;
```

Example -

```
#include<iostream>
using namespace std;
class Base {       //Abstract base class
   public:
   virtual void show() = 0;    // Pure Virtual Function
};
class Derived : public Base {
   public:
   void show()    {
       cout<<"Implementation of Virtual Function"<<endl;
   }
};
```

```
int main() {
   Base ob;   //Compile Time Error
   Base *b;
   Derived d;
   b = &d;
   b->show();
}
```

Output:

Implementation of Virtual Function

Note: *In the above example Base class is abstract, with pure virtual function* ***show()*** *, hence we cannot create object of base class.*

Member Classes (Nesting of Classes): A class can contain objects of other classes. This is known as containership or nesting. This can be shown as follows,

```
class Alpha { .......};
class Beta { .......};
class Gamma {
   .............
   Alpha a;
   Beta b;
  public:
   Gamma(arglist) : a(arglist1), b(arglist2)
   {
       ..... // constructor body
   }
};
```

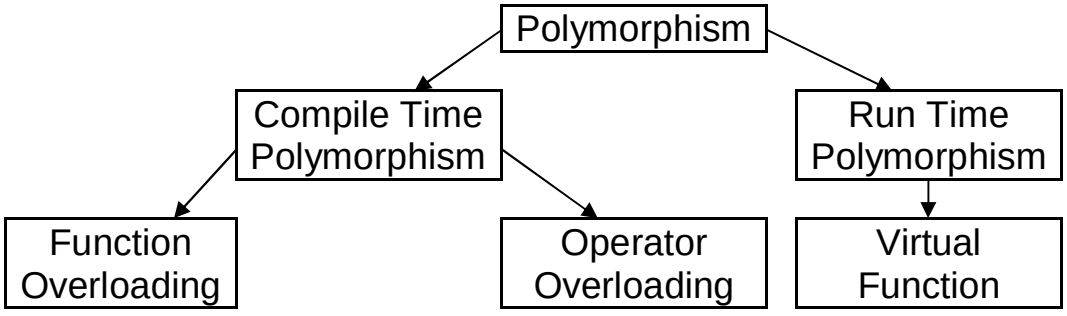
arglist is the list of arguments that is to be supplied when a **Gamma** object is defined. These parameters are used for initializing the members of **Gamma**. arglist1 is the argument list for the constructor of **a** and arglist2 is the argument list of the constructor of **b**. arglist1 and arglist2 may or may not use the arguments from arglist. a(arglist1) and b(arglist2) are function calls and therefore the arguments do not contain the data types. They are simply variables or constants.

Example-

```
Gamma(int x, int y, float z) : a(x), b(y,z) {
    Assignment section (for ordinary other members)
}
```

Polymorphism: The term "Polymorphism" is the combination of "poly" + "morphs" which means many forms. In other words, it means “one name, multiple forms”. Polymorphism is mainly divided into two types:

- Compile time Polymorphism
- Runtime Polymorphism (also known as late binding or dynamic binding)



Compile time polymorphism is achieved by function overloading or operator overloading.

Function Overloading: When there are multiple functions with same name but different parameters then these functions are said to be **overloaded**. Which function will be called depend on the parameters passed. Functions can be overloaded by **change in number of arguments** or/and **change in type of arguments**.

```
#include<iostream>
using namespace std;
class A {
   public:
   // function with 1 int parameter
   void func(int x)    {
      cout << "\nvalue of x is " << x;
   }
   //function with same name but 1 double parameter
   void func(double x)    {
      cout << "\nvalue of x is " << x;
   }
   // function with same name and 2 int parameters
   void func(int x, int y)    {
      cout<<"\nvalue of x and y are "<<x<<", "<<y;
   }
};
int main() {
   A obj1;
   obj1.func(7); // The 1st 'func' is called
   obj1.func(9.132); // 2nd 'func' is called
   obj1.func(85,64);  // 3rd 'func' is called
   return 0;
}
```

Output:

```
value of x is 7
value of x is 9.132
value of x and y is 85, 64
```

In this example, a single function named *func* acts differently in three different situations which is the property of polymorphism.

Operator Overloading: C++ also provides option to overload operators. For example, we can make the operator ('+') for string class to concatenate two strings. We know that this is the addition operator whose task is to add two operands. So a single operator '+' when placed between integer operands, adds them and when placed between string operands, concatenates them.

Example:

```
#include<iostream>
using namespace std;
class Complex {
   int real, imag;
public:
   Complex(int r = 0, int i =0)  {real = r;   imag = i;}
   Complex operator + (Complex obj) {
        Complex res;
        res.real = real + obj.real;
        res.imag = imag + obj.imag;
        return res;
   }
   void print() { cout<<real<<" + i"<<imag<<endl; }
};
int main() {
   Complex c1(10, 5), c2(2, 4);
   Complex c3 = c1 + c2; // call "operator+"
   c3.print();
}
```

Output: 12 + i9

In this example the operator '+' is overloaded. The operator '+' is an addition operator and can add two numbers(integers or floating point) but here the operator is made to perform addition of two imaginary or complex numbers.

Rules for overloading operators:

- Only existing operators can be overloaded. New operators cannot be created.
- The overloaded operator must have at least one operand that is of user defined type.
- We cannot change the basic meaning of an operator. That is to say, We cannot redefine the plus(+) operator to subtract one value from the other.
- Overloaded operators follow the syntax rules of the original operators. They cannot be overridden.

- There are **some operators that cannot be overloaded** like size of operator(**sizeof**), membership operator(**.**), pointer to member operator(**.***), scope resolution operator(**::**), conditional operators(**?:**) etc
- We **cannot use "friend" functions to overload certain operators**. However, member function can be used to overload them. Friend Functions cannot be used with assignment operator(=), function call operator(()), subscripting operator([ ]), class member access operator (->) etc.
- Unary operators, overloaded by means of a member function, take no explicit arguments and return no explicit values, but, those overloaded by means of a friend function, take one reference argument (the object of the relevant class).
- Binary operators overloaded through a member function take one explicit argument and those which are overloaded through a friend function take two explicit arguments.
- When using binary operators overloaded through a member function, the left hand operand must be an object of the relevant class.
- Binary arithmetic operators such as +,-,* and / must explicitly return a value. They must not attempt to change their own arguments.

Runtime polymorphism: This type of polymorphism is achieved by Function Overriding. **Function overriding** on the other hand occurs when a derived class has a definition for one of the member functions of the base class. That base function is said to be **overridden**. Runtime polymorphism is achieved through virtual function.

Virtual Functions: When we use the same function in both the base and derived classes, the function in base class is declared as **virtual** using the keyword **virtual** preceding its normal declaration. When a function is made **virtual**, C++ determines which function to use at run time based on the type of object pointed to by the base pointer, rather than the type of the pointer. Thus, by making the base pointer to point to different objects, we can execute different versions of the **virtual** function.

Example-

```
#include<iostream>
using namespace std;
class Base {
public:
   virtual void print ()
     { cout<< "\n print Base class"; }
   void show ()
     { cout<< "\n show Base class"; }
};
class Derived : public Base {
public:
   void print ()
     { cout<< "\n print Derived class"; }
   void show ()
     { cout<< "\n show Derived class"; }
};
```

```
int main()  {
   Base b;    Derived d;    Base *bptr;
   bptr = &b;
   bptr->print();    // calls Base version
   bptr->show();  // calls Base version
   bptr = &d;
   //virtual function, binded at runtime
   bptr->print();   //calls Derived version.
   // Non-virtual function, binded at compile time
   bptr->show(); // calls Base version
   return 0;
}
```

Output:

```
print Base class
show Base class
print Derived class
show Base class
```

Rules for Virtual Functions:

- Virtual functions must be members of some class.
- Virtual functions cannot be static members.
- They are accessed through object pointers.
- They can be a friend of another class.
- A virtual function must be defined in the base class, even though it is not used.
- The prototypes of a virtual function of the base class and all the derived classes must be identical. If the two functions with the same name but different prototypes, C++ will consider them as the overloaded functions.
- We cannot have a virtual constructor, but we can have a virtual destructor
- While a base pointer can point to any type of the derived object, the reverse is not true. That is, we cannot use a pointer to a derived class to access an object of the base type.
- When a base pointer points to a derived class, incrementing or decrementing it will not make it to point the next object of the derived class. It is incremented or decremented only to its base type. Therefore, we should not use this method to move the pointer to the next object.
- If a virtual function is defined in the base class, it need not be necessarily redefined in the derived class. In such cases, calls will invoke the base function.

Differences between compile time and run time polymorphism

Compile time polymorphism	Run time polymorphism
The function to be invoked is known at the compile time.	The function to be invoked is known at the run time.
It is also known as overloading, early binding and static binding.	It is also known as overriding, Dynamic binding or late binding.
Overloading is a compile time polymorphism where more than one method is having the same name but with the different number of parameters or the type of the parameters.	Overriding is a run time polymorphism where more than one method is having the same name, number of parameters and the type of the parameters.
It is achieved by function overloading and operator overloading.	It is achieved by virtual functions and pointers.
It provides fast execution as it is known at the compile time.	It provides slow execution as it is known at the run time.

Pointers: A **pointer** is a variable whose value is the address of another variable. The general form of a pointer variable declaration is − **type *var-name;**

Here, **type** is the pointer's base type; it must be a valid C++ type and **var-name** is the name of the pointer variable. Following are the valid pointer declaration −

```
int    *ip;                    // pointer to an integer
double *dp;        // pointer to a double
```

```
float  *fp;          // pointer to a float
char   *ch           // pointer to character
```

The actual data type of the value of all pointers, whether integer, float, character, or otherwise, is the same, a long hexadecimal number that represents a memory address. The only difference between pointers of different data types is the data type of the variable or constant that the pointer points to.

Advantage of pointer

1) Pointer reduces the code and improves the performance.
2) We can return multiple values from function using pointer.
3) It helps to access any memory location in the computer's memory.

Usage of pointer

1) Dynamic memory allocation
2) Pointers are widely used in arrays, functions and structures. It reduces the code and improves the performance.

Symbols used in pointer

Symbol	Name	Description
& (ampersand sign)	Address operator	Determine the address of a variable.
∗ (asterisk sign)	Indirection operator	Access the value of an address.

Pointer Arithmetic: As pointer is an address which is a numeric value; therefore, you can perform arithmetic operations on a pointer just as you can a numeric value. There are four arithmetic operators that can be used on pointers: ++, --, +, and -

To understand pointer arithmetic, let us consider that **ptr** is an integer pointer which points to the address 1000. Assuming 32-bit integers, let us perform the following arithmetic operation on the pointer − **ptr++;**

The **ptr** will point to the location 1004 because each time ptr is incremented, it will point to the next integer. This operation will move the pointer to next memory location without impacting actual value at the memory location. If ptr points to a character whose address is 1000, then above operation will point to the location 1001 because next character will be available at 1001.

Pointer Comparisons: Pointers may be compared by using relational operators, such as ==, <, <=, > and >=. If p1 and p2 point to variables that are related to each other, such as elements of the same array, then p1 and p2 can be meaningfully compared.

Void Pointers: Void pointer or **generic pointer** is a special type of pointer that can be pointed at objects of any data type. A void pointer is declared like a normal pointer, using the void keyword as the pointer's type. Pointers defined using specific data type cannot hold the address of the some other type of variable i.e., it is incorrect in C++ to assign the address of an

Example-

```
void *v;     int *i;
int ivar;    float fvar;     c
har chvar;
v = &ivar;          // valid
v = &chvar;         //valid
v = &fvar;          // valid
i = &ivar;          // valid
```

integer variable to a pointer of type float. But void pointers can hold address of any data type.

```
i = &chvar;     // invalid
i = &fvar;      // invalid
```

Wild pointers: A pointer behaves like a wild pointer when it is declared but not initialized. In this example, a pointer x is declared but not initialized. So, it is displaying contents of some random memory locations.

```
#include<iostream>
using namespace std;
int main()  {    int *x;
for(int k=0; k<3; k++)    cout<<x[k]<<"\t";
   return 0;   }
```

Output:

```
28005    29739    29541
```

Dangling pointer: Generally, daggling pointers arise when the referencing object is deleted or deallocated, without changing the value of the pointers. It creates the problem because the pointer is still pointing to the memory that is not available. When the user tries to dereference the daggling pointers than it shows the undefined behaviour and can be the cause of the segmentation fault. For example, a pointer to a local variable after the variable goes out of scope.

```
#include<iostream>
using namespace std;
int main() {
   int * p;
   {                  //block
          int Data = 27;
          p = &Data;
   }
   cout<<"p_Data="<< *p; //p is dangling pointer
   return 0;
}
```

Pointers to Objects: A pointer to object in C++ is done exactly the same way as a pointer to a structure and to access members of a pointer to a class we use the member access operator - >, just as we do with pointers to structures. Also as with all pointers, we must initialize the pointer before using it. This is illustrated in the following example.

```
#include<iostream>
using namespace std;
class Box {
     double length, breadth, height;
  public:
       Box(double l, double b, double h) {
       cout <<"\nConstructor called.";
       length=l;
       breadth=b;
       height=h;
     }
     double Volume() {
       return length * breadth * height;
     }
};
```

```
void main() {
   Box Box1(3.3, 1.2, 1.5);    // Declare box1
   Box Box2(8.5, 6.0, 2.0);    // Declare box2
   Box *ptrBox;       // Declare pointer to a class.
   ptrBox = &Box1;
   cout<<"\nVolume of Box1: "<<ptrBox->Volume();
   ptrBox = &Box2;
   cout<<"\nVolume of Box2: "<<ptrBox->Volume();
}
```

Output:

```
Constructor called.
Constructor called.
Volume of Box1: 5.94
Volume of Box2: 102
```

this Pointer: The **this** pointer holds the address of current object, in simple words you can say that this **pointer** points to the current object of the class. Let's take an example to understand this concept.

```
#include<iostream>
using namespace std;
class Box {
        double length, breadth, height;
   public:
        Box(double l, double b, double h) {
        cout <<"\nConstructor called.";
        length=l;    breadth=b;    height=h;  }
      double Volume() {
        return length * breadth * height;      }
      int compare(Box box) {
        return this->Volume()>box.Volume();
     }  };
```

```
int main(void) {
   Box Box1(3.3, 1.2, 1.5);    // Declare box1
   Box Box2(8.5, 6.0, 2.0);    // Declare box2
   if(Box1.compare(Box2))
      cout << "\nBox1 is bigger than Box2";
   else
    cout<<"\nBox1 is equal to or smaller than Box2";
   return 0;
}
```

Output −

```
Constructor called.
Constructor called.
Box1 is equal to or smaller than Box2
```

Pointers to Derived Classes: A base class pointer can point to objects of both base and derived class; whereas a derived class pointer cannot point to the object of the base class.

```
#include<iostream>
using namespace std;
class Base {
    public:
        int x;
        Base(int a) { x=a; }
        void bShow() {    cout<<"\nX = "<<x;    }
};
class Derive: public Base {
            public:
            int y;
            Derive(int a, int b) { x=a; y=b;)
            void dShow ()   {
                cout<<"\nX = "<<x<<"\tY = "<<y<<endl;
            }
};
```

```
int main ()  {
        Base B1(10), *bptr;
        Derive D1(20, 30), *dptr;

        bptr = &B1;       bptr->bShow();
        bptr = &D1;       bptr->dShow();

        dptr = &D1;         dptr->dShow ();

       // dptr = &B1;   Error
}
```

Output:

```
X = 10
X = 20       Y = 30
X = 20       Y = 30
```